Kanbien Platform Adapter Consumption Model Guide

Predictable consumption patterns for platform adapters, backwards compatibility, and the Option B one-package-per-adapter architecture.

Prepared for the Kanbien architecture guides
Version 1.0
Generated 2026-06-22

Decision: each platform adapter package should expose a predictable factory-based API, return stable packages/core contracts, and be consumed by applications only through bootstrap or platform profile boundaries.

Table of Contents

1. Purpose of this guide

This guide explains how Kanbien application layers should consume platform functionality in a predictable, backwards-compatible way. It builds on the decision to use **Option B: one package per adapter** for future scalability and evolution.

The core problem is simple: once Kanbien has many platform adapters, each adapter must be easy to replace, easy to version, and hard to misuse from product code.

The architectural aim is not multi-cloud complexity on day one. The aim is clean provider separation, stable app-facing contracts, and a migration path that does not require product logic rewrites.

The central question

Should each platform functionality layer have a predictable way of being consumed by application layers for backwards compatibility?

Yes. Every platform adapter should expose a predictable consumption model. But application feature code should consume **Kanbien core contracts**, not vendor SDKs or adapter implementation details.

The short answer

```
Core interface:
  packages/core/files/FileStorage

Adapter package:
  @kanbien/platform-aws-files-s3
  createS3FileStorage(config): FileStorage

Profile package:
  @kanbien/platform-profile-aws-standard
  createAwsStandardPlatform(config): ApiPlatform / WorkerPlatform

Application feature code:
  consumes FileStorage, EventBus, Queue, AuditService, etc.

Application bootstrap:
  consumes the selected platform profile.
```

2. The core rule

For Kanbien, use this rule:

```
Feature and application logic consumes packages/core interfaces.

App bootstrap wires those interfaces to platform adapters or platform profiles.

Platform adapters expose predictable factory functions and config contracts.
```

This means application services should depend on abstractions such as `FileStorage`, `Queue`, `EventBus`, `NotificationService`, `AuditService`, `ConfigProvider`, `Logger`, and `Metrics`.

Application services should not depend directly on vendor-specific clients such as `S3Client`, `SQSClient`, `EventBridgeClient`, `AzureBlobClient`, or `RabbitMqConnection`.

Dependency direction

```
apps/feature-code
  -> packages/core contracts only

apps/bootstrap
  -> platform profiles or selected adapters

platform/profiles
  -> individual adapter packages

platform/adapters
  -> packages/core contracts + vendor SDKs

packages/core
  -> no dependency on platform, apps, or infra
```

Why this protects backwards compatibility

Backwards compatibility is preserved at two boundaries:

- The **core contract boundary**, where application logic depends on stable Kanbien interfaces.
- The **platform profile boundary**, where application bootstrap receives a stable platform object even if underlying adapters change.

If Kanbien later moves file storage from S3 to Azure Blob Storage, the product service should not change if it only depends on `FileStorage`.

3. Why predictable consumption matters

Option B gives Kanbien one package per adapter. This is scalable, but it can become chaotic if every adapter exposes a different API shape.

The risk without a standard

```
// Bad: every adapter has its own shape

const files = new S3StorageClient(...)
const queue = SqsAdapter.connect(...)
const events = makeEventBridge(...)
const notifications = await SesProvider.initialize(...)
```

This creates inconsistent bootstrap code, makes migrations harder, and increases the odds that product logic starts importing vendor-specific code.

The preferred standard

```
// Good: every adapter exposes a predictable factory

const files = createS3FileStorage(config.files)
const queue = createSqsQueue(config.queue)
const events = createEventBridgeEventBus(config.events)
const notifications = createSesNotificationService(config.notifications)
```

Each factory returns a stable `packages/core` interface. The adapter may use AWS, Azure, or open-source technology internally, but the consuming application sees a Kanbien contract.

The payoff

- Applications remain stable while platform implementations evolve.
- Adapters can be versioned independently.
- Providers can be swapped behind profile boundaries.
- Feature code stays free of vendor SDK imports.
- Testing is easier because services can be replaced with local or fake implementations.

4. Recommended consumption levels

Kanbien should have three clear consumption levels. Each level is allowed to know a different amount about the runtime environment.

Level 1: feature code

Feature code should consume only core contracts. It should not know whether Kanbien is running on AWS, Azure, open-source infrastructure, or local test implementations.

```
import type { EventBus } from "@kanbien/core/events";
import type { AuditService } from "@kanbien/core/audit";

export function createUserService(deps: {
  eventBus: EventBus;
  audit: AuditService;
}) {
  return {
    async createUser(input: CreateUserInput) {
      // Product logic goes here.
      // No AWS, Azure, S3, SQS, EventBridge, or vendor SDK code.
    },
  };
}
```

Level 2: app bootstrap

App bootstrap is allowed to know which platform profile is being used. This is the composition boundary where runtime dependencies are wired together.

```
import { createAwsStandardPlatform } from "@kanbien/platform-profile-aws-standard";
import { createAppServices } from "../services";

const platform = createAwsStandardPlatform(config.platform);

export const services = createAppServices({
  files: platform.files,
  queue: platform.queue,
  eventBus: platform.eventBus,
  notifications: platform.notifications,
  metrics: platform.metrics,
  logger: platform.logger,
});
```

Good locations for this code include `apps/api/src/bootstrap/platform.ts`, `apps/worker/src/bootstrap/platform.ts`, and `apps/admin/src/bootstrap/platform.ts`.

Level 3: platform profiles

Platform profiles compose individual adapter packages into a stable app-facing platform object.

```
import { createS3FileStorage } from "@kanbien/platform-aws-files-s3";
import { createSqsQueue } from "@kanbien/platform-aws-queues-sqs";
import { createEventBridgeEventBus } from "@kanbien/platform-aws-events-eventbridge";
import { createSesNotificationService } from
  "@kanbien/platform-aws-notifications-ses";

export function createAwsStandardPlatform(config: AwsStandardPlatformConfig):
  ApiPlatform {
  return {
    files: createS3FileStorage(config.files),
    queue: createSqsQueue(config.queue),
    eventBus: createEventBridgeEventBus(config.events),
    notifications: createSesNotificationService(config.notifications),
  };
}
```

This is where Option B pays off. You get fine-grained adapter packages without making each app wire every individual provider dependency by hand.

5. Option B: one package per adapter

Kanbien has chosen Option B: one package per adapter. This prepares the platform for future scalability, modularity, and provider evolution.

What Option B means

```
platform/aws/files-s3/
platform/aws/queues-sqs/
platform/aws/events-eventbridge/
platform/aws/notifications-ses/
platform/aws/authn-cognito/
platform/aws/security-kms/

platform/azure/files-blob/
platform/azure/queues-service-bus/
platform/azure/events-event-grid/

platform/oss/files-minio/
platform/oss/queues-rabbitmq/
platform/oss/events-nats/
```

Why this is the right decision for Kanbien

- It isolates vendor SDK dependencies by adapter.
- It allows independent versioning of each implementation.
- It lets Kanbien compose different deployment profiles from smaller building blocks.
- It reduces blast radius when one adapter changes.
- It supports future enterprise deployment patterns without rewriting core or apps.

The tradeoff

Option B creates more packages and more boilerplate. That is real overhead. Kanbien should manage this by using clear naming conventions, standard adapter package shapes, and profile packages that make consumption simple for apps.

Option B should not mean apps import fifteen adapter packages directly. Apps should usually import one profile package, and profiles should compose the adapters.

6. Standard adapter package contract

Each platform adapter package should expose the same public shape. The exact implementation can differ, but the consumption model should feel familiar across all adapters.

Required exports

Export	Purpose
Factory function	Creates the adapter and returns a packages/core interface.
Config type	Documents required configuration fields.
Config schema	Validates runtime configuration before the adapter starts.
Health check	Allows runtime readiness and diagnostics.
Stable named exports	Prevents unclear or unstable import patterns.

Naming convention

```
createS3FileStorage(config): FileStorage
createAzureBlobFileStorage(config): FileStorage
createMinioFileStorage(config): FileStorage

createSqsQueue(config): Queue
createAzureServiceBusQueue(config): Queue
createRabbitMqQueue(config): Queue

createEventBridgeEventBus(config): EventBus
createEventGridEventBus(config): EventBus
createNatsEventBus(config): EventBus
```

Example package shape

```
platform/aws/files-s3/
  package.json
  README.md
  src/
    index.ts
    config.ts
    s3-file-storage.ts
    s3-signed-url-service.ts
    health.ts
    errors.ts
    testing.ts
```

Example public exports

```
// platform/aws/files-s3/src/index.ts

export {
  createS3FileStorage,
  type S3FileStorageConfig,
  s3FileStorageConfigSchema,
} from "./s3-file-storage";

export {
  createS3SignedUrlService,
  type S3SignedUrlServiceConfig,
} from "./s3-signed-url-service";

export {
  checkS3FileStorageHealth,
} from "./health";
```

Example implementation

```
import type { FileStorage } from "@kanbien/core/files";

export interface S3FileStorageConfig {
  region: string;
  bucketName: string;
}

export function createS3FileStorage(
  config: S3FileStorageConfig
): FileStorage {
  return new S3FileStorage(config);
}
```

The key detail is that `createS3FileStorage` returns `FileStorage`, not an AWS-specific type.

7. Platform profiles

Platform profiles are the missing piece that keep Option B usable. They compose many adapter packages into a stable platform object for a specific deployment style.

Why profiles exist

Without profiles, every app would need to import and configure many adapter packages. That is repetitive and error-prone.

```
// Avoid making every app do this manually

import { createS3FileStorage } from "@kanbien/platform-aws-files-s3";
import { createSqsQueue } from "@kanbien/platform-aws-queues-sqs";
import { createEventBridgeEventBus } from "@kanbien/platform-aws-events-eventbridge";
import { createSesNotificationService } from
  "@kanbien/platform-aws-notifications-ses";
import { createCloudWatchMetrics } from "@kanbien/platform-aws-monitoring-cloudwatch";
```

Profiles centralize this composition and expose a smaller, stable app-facing surface.

Recommended profile packages

```
platform/profiles/local-dev/
platform/profiles/test/
platform/profiles/aws-standard/
platform/profiles/aws-enterprise/
platform/profiles/azure-standard/
platform/profiles/oss-local/
```

Example profile contract

```
export interface ApiPlatform {
  logger: Logger;
  metrics: Metrics;
  authenticator: Authenticator;
  authorizer: Authorizer;
  audit: AuditService;
  eventBus: EventBus;
  files: FileStorage;
  notifications: NotificationService;
}

export interface WorkerPlatform {
  logger: Logger;
  metrics: Metrics;
  queue: Queue;
  jobs: JobScheduler;
  eventBus: EventBus;
  notifications: NotificationService;
  audit: AuditService;
}
```

A public API app, a background worker, and a frontend app should not all receive the same giant platform object. Each app type should receive the platform surface it actually needs.

Example AWS standard profile

```
export interface AwsStandardPlatformConfig {
  files: S3FileStorageConfig;
  queue: SqsQueueConfig;
  events: EventBridgeEventBusConfig;
  notifications: SesNotificationServiceConfig;
  monitoring: CloudWatchMetricsConfig;
}

export function createAwsStandardPlatform(
  config: AwsStandardPlatformConfig
): ApiPlatform {
  const logger = createStdoutJsonLogger();

  return {
    logger,
    metrics: createCloudWatchMetrics(config.monitoring),
    authenticator: createCognitoAuthenticator(config.authn),
    authorizer: createVerifiedPermissionsAuthorizer(config.authz),
    audit: createPostgresAuditService(config.audit),
    eventBus: createEventBridgeEventBus(config.events),
    files: createS3FileStorage(config.files),
    notifications: createSesNotificationService(config.notifications),
  };
}
```

8. Backwards compatibility strategy

Backwards compatibility should be planned at the core interface, adapter package, and profile package levels.

1. Stable core interfaces

Core interfaces should change slowly and deliberately. They are the most important compatibility boundary.

```
export interface FileStorage {
  put(input: PutFileInput): Promise<StoredFile>;
  get(id: string): Promise<FileObject>;
  delete(id: string): Promise<void>;
}
```

Prefer additive changes. But be careful with optional methods. Too many optional methods can make an interface weak and unpredictable.

Use capability interfaces instead of bloated universal interfaces

```
export interface FileStorage {
  put(input: PutFileInput): Promise<StoredFile>;
  get(id: string): Promise<FileObject>;
  delete(id: string): Promise<void>;
}

export interface SignedUrlService {
  createDownloadUrl(input: CreateDownloadUrlInput): Promise<SignedUrl>;
}

export interface MultipartUploadService {
  begin(input: BeginMultipartUploadInput): Promise<MultipartUpload>;
  uploadPart(input: UploadPartInput): Promise<void>;
  complete(input: CompleteMultipartUploadInput): Promise<StoredFile>;
}
```

2. Standard config pattern

Every adapter should have an explicit config type and a validation schema.

```
export interface SqsQueueConfig {
  region: string;
  queueUrl: string;
  visibilityTimeoutSeconds?: number;
  maxReceiveCount?: number;
}

export const sqsQueueConfigSchema = z.object({
  region: z.string().min(1),
  queueUrl: z.string().url(),
  visibilityTimeoutSeconds: z.number().int().positive().optional(),
  maxReceiveCount: z.number().int().positive().optional(),
});
```

3. Stable profile contracts

Profiles should expose stable app-facing objects such as `ApiPlatform` and `WorkerPlatform`. The underlying implementation can change as long as the returned contract remains compatible.

4. Semantic versioning

With Option B, each adapter can version independently.

```
@kanbien/platform-aws-files-s3@1.4.0
@kanbien/platform-aws-queues-sqs@1.2.0
@kanbien/platform-aws-events-eventbridge@2.0.0
```

Version change	Meaning
Patch	Bug fixes with no public API change.
Minor	Backwards-compatible additions.
Major	Breaking changes that require migration.

5. Deprecation policy

Never silently change adapter behaviour. Use a clear deprecation path.

```
1. Add the new method or config option.
2. Mark the old method or config option deprecated.
3. Support both for a defined period.
4. Provide migration notes.
5. Remove the old path in the next major version.
```

9. Compatibility between providers

Kanbien does not need every provider to support every feature identically. That is a trap. Providers have different semantics for ordering, retries, visibility timeouts, dead-lettering, delivery guarantees, message size, fanout, and consumer groups.

Avoid universal interfaces that leak vendor concepts

```
// Bad
interface UniversalStorage {
  put(): Promise<void>;
  get(): Promise<void>;
  createS3PresignedPost(): Promise<void>;
  createAzureSasToken(): Promise<void>;
}
```

Prefer capability interfaces

```
// Good
interface FileStorage {
  put(input: PutFileInput): Promise<StoredFile>;
  get(id: string): Promise<FileObject>;
  delete(id: string): Promise<void>;
}

interface SignedUrlService {
  createDownloadUrl(input: CreateDownloadUrlInput): Promise<SignedUrl>;
}
```

Providers then implement the capabilities they support. For example, the S3 adapter may implement `FileStorage`, `SignedUrlService`, and `MultipartUploadService`. The local file adapter may implement only `FileStorage`.

Compatibility should be honest

Do not hide important provider differences behind a misleading interface. The core interface should model what Kanbien actually needs, not pretend all providers behave the same.

10. Recommended repository structure

This structure combines Option B with predictable consumption and platform profiles.

```
kanbien/
  packages/
    core/
      src/
        authn/
        authz/
        files/
        queues/
        events/
        audit/
        notifications/
        logging/
        monitoring/
        config/
        tenancy/
        persistence/
        security/
        analytics/
        reporting/
        async-jobs/
        i18n/
        localization/
        runtime/
          api-platform.ts
          worker-platform.ts

  platform/
    shared/
      retry/
      serialization/
      observability/
      idempotency/
      errors/

    local/
      files-local/
      queues-in-memory/
      events-in-memory/
      notifications-console/
      authn-fake/
      authz-fake/
      logging-stdout/
      monitoring-noop/

    aws/
      files-s3/
      queues-sqs/
      events-eventbridge/
      async-jobs-sqs/
      notifications-ses/
      authn-cognito/
      authz-verified-permissions/
      config-appconfig/
      config-secrets-manager/
      logging-cloudwatch/
      monitoring-cloudwatch/
      security-kms/
      audit-postgres/
      analytics-firehose/
      reporting-s3/

    azure/
      files-blob/
      queues-service-bus/
      events-event-grid/
      notifications-communication-services/
      authn-entra-external-id/
      config-app-configuration/
      security-key-vault/

    oss/
      files-minio/
```

```
      queues-rabbitmq/
      events-nats/
      authn-keycloak/
      authz-openfga/
      config-unleash/
      security-vault/
      monitoring-prometheus/
      logging-loki/

    profiles/
      local-dev/
      aws-standard/
      aws-enterprise/
      azure-standard/
      oss-local/
```

What belongs in packages/core/runtime

If multiple apps need the same platform surface, define shared platform-facing types in `packages/core/runtime`. These should be app-facing contracts, not provider implementations.

```
// packages/core/runtime/api-platform.ts

export interface ApiPlatform {
  logger: Logger;
  metrics: Metrics;
  authenticator: Authenticator;
  authorizer: Authorizer;
  audit: AuditService;
  eventBus: EventBus;
  files: FileStorage;
  notifications: NotificationService;
}
```

What belongs in platform/shared

Use `platform/shared` only for provider-neutral implementation helpers, not Kanbien business meaning.

Belongs in platform/shared	Does not belong in platform/shared
Retry helpers	TenantId
Serialization helpers	Principal
Idempotency helpers	Permission definitions
Adapter error base classes	Domain events such as UserCreated
Observability context helpers	Product workflows

11. Example: files with S3

This worked example shows how a file upload capability should flow from core contract to adapter to platform profile to application service.

Core contract

```
// packages/core/files/file-storage.ts

export interface PutFileInput {
  filename: string;
  contentType: string;
  sizeBytes: number;
  body: AsyncIterable<Uint8Array>;
  tenantId?: string;
}

export interface StoredFile {
  id: string;
  filename: string;
  contentType: string;
  sizeBytes: number;
  tenantId?: string;
  createdAt: Date;
}

export interface FileObject extends StoredFile {
  body: AsyncIterable<Uint8Array>;
}

export interface FileStorage {
  put(input: PutFileInput): Promise<StoredFile>;
  get(id: string): Promise<FileObject>;
  delete(id: string): Promise<void>;
}
```

AWS adapter

```
// platform/aws/files-s3/src/s3-file-storage.ts

import type { FileStorage } from "@kanbien/core/files";

export interface S3FileStorageConfig {
  region: string;
  bucketName: string;
}

export function createS3FileStorage(
  config: S3FileStorageConfig
): FileStorage {
  return new S3FileStorage(config);
}

class S3FileStorage implements FileStorage {
  constructor(private readonly config: S3FileStorageConfig) {}

  async put(input: PutFileInput): Promise<StoredFile> {
    // Use AWS SDK internally.
    // Return the Kanbien StoredFile contract.
  }

  async get(id: string): Promise<FileObject> {
    // Use AWS SDK internally.
  }

  async delete(id: string): Promise<void> {
    // Use AWS SDK internally.
  }
}
```

Profile composition

```
// platform/profiles/aws-standard/src/platform.ts

export function createAwsStandardPlatform(
  config: AwsStandardPlatformConfig
): ApiPlatform {
  return {
    files: createS3FileStorage(config.files),
    eventBus: createEventBridgeEventBus(config.events),
    audit: createPostgresAuditService(config.audit),
    logger: createStdoutJsonLogger(),
    metrics: createCloudWatchMetrics(config.monitoring),
    notifications: createSesNotificationService(config.notifications),
    authenticator: createCognitoAuthenticator(config.authn),
    authorizer: createVerifiedPermissionsAuthorizer(config.authz),
  };
}
```

Application service

```
// apps/api/src/documents/upload-document.ts

import type { FileStorage } from "@kanbien/core/files";
import type { AuditService } from "@kanbien/core/audit";

export function createUploadDocumentService(deps: {
  files: FileStorage;
  audit: AuditService;
}) {
  return {
    async upload(input: UploadDocumentInput) {
      const stored = await deps.files.put({
        filename: input.filename,
        contentType: input.contentType,
        sizeBytes: input.sizeBytes,
        body: input.body,
        tenantId: input.tenantId,
      });

      await deps.audit.record({
        action: "file.uploaded",
        actor: { id: input.actorId, type: "user" },
        tenantId: input.tenantId,
        resource: { type: "file", id: stored.id },
        occurredAt: new Date(),
      });

      return stored;
    },
  };
}
```

The application service knows nothing about S3. That is the point.

12. Do and don't rules

Do	Do not
Consume packages/core interfaces in feature code.	Import AWS, Azure, or OSS SDKs from feature code.
Use platform profiles in app bootstrap.	Make every feature service configure its own adapters.
Expose predictable createX(config) factories.	Use inconsistent names such as init, build, make, connect randomly.
Validate adapter config at startup.	Wait for config mistakes to fail during user requests.
Use capability interfaces for optional provider features.	Create universal interfaces that leak vendor-specific concepts.
Version adapter packages independently.	Force unrelated adapters to change version together.
Keep platform/shared boring.	Let platform/shared become a second core package.

Allowed imports

```
// Allowed in feature code
import type { FileStorage } from "@kanbien/core/files";
import type { EventBus } from "@kanbien/core/events";

// Allowed in app bootstrap
import { createAwsStandardPlatform } from "@kanbien/platform-profile-aws-standard";

// Allowed in platform profile packages
import { createS3FileStorage } from "@kanbien/platform-aws-files-s3";

// Allowed inside adapter implementation packages
import { S3Client } from "@aws-sdk/client-s3";
```

Disallowed imports

```
// Bad: feature code directly imports AWS adapter
import { createS3FileStorage } from "@kanbien/platform-aws-files-s3";

// Bad: feature code directly imports vendor SDK
import { S3Client } from "@aws-sdk/client-s3";

// Bad: packages/core imports platform implementation
import { createSqsQueue } from "@kanbien/platform-aws-queues-sqs";
```

13. Architecture decision record

Kanbien should formalize this as an architecture decision record.

ADR: Platform adapter consumption model

Field	Decision
Status	Accepted
Context	Kanbien uses Option B: one package per platform adapter. This creates long-term modularity but requires predictable consumption patterns to avoid app-layer complexity and vendor coupling.
Decision	Each platform adapter package must implement one or more packages/core contracts and expose a predictable factory-based API. Application feature code must depend only on packages/core contracts. App bootstrap may depend on platform profile packages. Platform profile packages compose individual adapter packages into stable app-facing platform objects.
Consequences	Provider implementations can evolve independently. Apps avoid direct vendor coupling. Backwards compatibility is managed at the core contract and profile boundary. One-package-per-adapter remains scalable without making app bootstrapping chaotic.

Implementation notes

- Create `packages/core/runtime/api-platform.ts` and `packages/core/runtime/worker-platform.ts` only if multiple apps need stable shared platform surfaces.
- Start with `platform/local`, `platform/aws`, `platform/shared`, and `platform/profiles/aws-standard`.
- Do not implement `platform/azure` or `platform/oss` until there is a real use case, but reserve the structure in architecture docs.
- Enforce import boundaries through lint rules or monorepo dependency constraints.

14. Final principles

The platform adapter model should optimize for clarity, compatibility, and future evolution. The following principles should guide Kanbien's implementation.

1. Core contracts are the app-facing truth

Applications should express what they need using Kanbien contracts. They should not express how those needs are implemented in a particular cloud provider.

2. Adapter packages should be small and replaceable

An adapter should implement a narrow capability. It should be easy to test, version, and replace.

3. Profiles make Option B usable

One package per adapter is powerful, but too granular for most app bootstrap code. Platform profiles give apps a simple consumption surface while preserving adapter modularity underneath.

4. Backwards compatibility lives at the boundaries

The two most important stability boundaries are the core interfaces and the platform profile contracts. Protect them carefully.

5. Provider differences should be explicit

Do not pretend SQS, Service Bus, RabbitMQ, NATS, Kafka, S3, Blob Storage, and MinIO are identical. Model the Kanbien capabilities you need and expose provider-specific differences only where they are real and necessary.

6. Product code should remain boring

A product engineer working on users, files, audit, or notifications should not need to understand the current cloud provider. They should need to understand Kanbien's core contracts.

Final recommendation: use Option B with standard adapter factories, stable config schemas, profile packages, core runtime contracts where useful, and strict import boundaries. This gives Kanbien long-term scalability without letting platform complexity leak into application logic.